

Verified Generators and Relations for Qupit Clifford Operators

ANONYMOUS AUTHOR(S)

Bian, Li, Ross, van de Wetering, and Zhao give sixteen rewrite rules that are sound and complete for multi-qudit Clifford circuits over H , S , CZ in every odd prime dimension p . We report a machine-checked proof of the result in safe, axiom-free Agda, against an independently constructed semantics. The proof follows the paper’s for the symplectic factor, where the paper does its real work, and diverges once, at the step that promotes symplectic completeness to projective completeness “up to Paulis”: there we factor the projective Clifford group as $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$, present the factors separately, and glue the presentations with a generic, verified semidirect-product theorem. The symplectic factor carries the mathematics: the paper’s normal form, refined into a doubly inductive datatype of *normal boxes* and *rows*, is a tower of verified Reidemeister–Schreier coset tables, proved unique by induction on the width, and the paper’s 42 box relations are derived from eighteen natural ones. Verified isomorphisms of presentations reach the paper’s own rules, and a central extension, verified against a cocycle, restores the scalars. Everything is parametric in p and typechecks with no postulates.

CCS Concepts: • **Theory of computation** → **Type theory; Logic and verification; Equational logic and rewriting; Quantum computation theory.**

Additional Key Words and Phrases: Clifford group, qudits, generators and relations, completeness, normal forms, symplectic group, semidirect product, Agda, formal verification

ACM Reference Format:

Anonymous Author(s). 2026. Verified Generators and Relations for Qupit Clifford Operators. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 1 (October 2026), 12 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

A finite list of circuit equations is *sound* when each holds in the intended semantics and *complete* when every semantic equality between circuits is derivable from it. Completeness is proved by a *normal form*, and the case analysis of every generator against every normal form is enormous: Selinger’s n -qubit Clifford presentation [Selinger 2015] runs a substantial rewrite system through it, its qutrit successor [Li et al. 2025] needs over fifty pages, and its generalisation to all odd primes by Bian, Li, Ross, van de Wetering, and Zhao [Bian et al. 2026a] — sixteen rules for multi-qudit Clifford circuits in every odd prime dimension p , henceforth “the qupit paper” — rests on roughly a hundred pages of derivations [Bian et al. 2026b].

We report a machine-checked proof of that theorem in safe, axiom-free Agda [Norell 2007; The Agda Community 2026], against a semantics constructed independently of the syntax, on the library of [Anonymous 2026a,b] (introduced in Appendix B). The proof follows the paper’s for the symplectic factor, where the paper does its real work, and diverges once, at the step that promotes symplectic to projective completeness *up to Paulis* (§2); there we instead *factor the group, present the factors, compose the presentations*: $\overline{C}_n = \overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$ and C_n is a central extension of it by the scalars; the symplectic group gets a coset-table normal form, the Pauli group a compositional presentation, the scalars a cyclic one, generic verified theorems for semidirect products and central extensions glue them, and plumbing reduces the result to the paper’s H , S , CZ and rules.

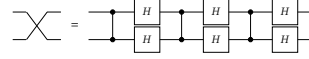
2 Qubit Clifford Circuits and the Main Theorems

Fix an odd prime p and $\omega = e^{2\pi i/p}$. On the basis $|0\rangle, \dots, |p-1\rangle$ indexed by $\mathbb{Z}_p$, $X|j\rangle = |j+1\rangle$ and $Z|j\rangle = \omega^j|j\rangle$ generate, with the phases ω^t , the Pauli group $\mathcal{P}_n$, and $H: |j\rangle \mapsto \frac{\delta_p}{\sqrt{p}} \sum_k \omega^{jk}|k\rangle$, $S: |j\rangle \mapsto \omega^{j(j-1)/2}|j\rangle$, $CZ: |j\rangle|l\rangle \mapsto \omega^{jl}|j\rangle|l\rangle$ generate, with the phases ω^t , the Clifford group C_n ($\delta_p = i^{(p-1)/2}$ makes the determinant of H equal to 1; the determinants of S and CZ are 1 too) [Bian et al. 2026a]. Modulo scalars, $\overline{C}_n = C_n/\langle\omega\rangle$ and $\overline{\mathcal{P}}_n \cong \mathbb{Z}_p^{2n}$, $C_n/\mathcal{P}_n \cong \overline{C}_n/\overline{\mathcal{P}}_n \cong \text{Sp}(2n, \mathbb{Z}_p)$, and $\overline{C}_n \cong \overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$. These “glue facts” are the two not formalised: our semantic groups are defined as the right-hand sides. In [Bian et al. 2026a] the global phases also include -1 , so that the swap and the multiplier gates belong to C_n ; we work mainly in the projective quotient, where -1 makes no difference except that the relations come out a little simpler.

Using the framework of Appendix B, we encode circuits, circuit relations, and reasoning with the relations. We draw circuits in matrix-multiplication order, the reverse of the usual circuit order: $w \bullet v$ applies v first but draws it on the right. For example, the swap:

Ex : Word (Gen $_{(2+, n)}$)

Ex = CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑ • CZ • H ↓ • H ↑

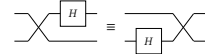


A rule set is an inductive family indexed by the width, one constructor per rule, each stated once at the smallest width at which it fits; for example, two of the fifteen constructors of our primary rule set $_SRel_===_$, the order of the swap and the rule moving H across it:

data $_SRel_===_ : (n : \mathbb{N}) \rightarrow \text{WRel} (\text{Gen } n)$ where

order-Ex : $(2+, n) \text{ SRel, Ex } \wedge 2 === \varepsilon$

semi-Ex-H ↑ : $(2+, n) \text{ SRel, Ex } \bullet H \uparrow === H \downarrow \bullet \text{Ex}$



And reasoning with the rules is equational reasoning in the congruence $\approx$ they generate, with combinators for symmetry, associativity, and rewriting the left or the right factor of a product; for example, conjugating by the swap moves H down a wire:

conj-Ex-H ↑ : $\text{Ex} \bullet H \uparrow \bullet \text{Ex} \approx H \downarrow$

conj-Ex-H ↑ = begin

Ex • H ↑ • Ex $\approx$ (sym assoc)

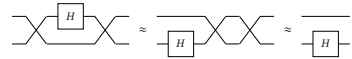
(Ex • H ↑) • Ex $\approx$ (cleft axiom semi-Ex-H ↑)

(H ↓ • Ex) • Ex $\approx$ (assoc)

H ↓ • Ex • Ex $\approx$ (right axiom order-Ex)

H ↓ • ε $\approx$ (right-unit)

H ↓ ■



Further, we can express and prove our main theorems:

THEOREM 2.1 (CLIFFORD-PRESENTATION). For every width n , the fifteen rules—the relations of [Bian et al. 2026a], read modulo scalars—present the projective Clifford group:

clifford-presentation : $\forall n \rightarrow (\text{PapV1.Clifford-Relations. } _QRel_===_ n) \text{ IsPresentationOf } (\text{Pauli} \rtimes \text{Sp } n)$

$_QRel_===_$ closes $_SRel_===_$ under the three structural rules (Appendix B); $\text{Pauli} \rtimes \text{Sp } n$ is the semidirect product of $\mathbb{Z}_p^{2n}$ by $\text{Sp}(2n, \mathbb{Z}_p)$ (§4); $_IsPresentationOf_$ (Appendix B) is a group isomorphism $\text{Cir } n / \approx \rightarrow \overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$: soundness (well-definedness of the map), each rule holds; completeness, $[[_]]$ is injective; and surjectivity, each projective Clifford operator is represented by a circuit.

We refine the qubit paper’s normal form for symplectic operators (Clifford operators modulo Paulis) into a doubly inductive definition (§3) and prove that it enjoys the unique-normal-form property:

THEOREM 2.2 (UNIQUE-NF). *Normal forms with equal symplectic denotations are equal:*

$$\text{unique-nf} : \forall n \{u \ v : NF\ n\} \rightarrow [[[u]]] \approx^s [[[v]]] \rightarrow u \equiv v$$

The route. We mostly follow the paper: its normal boxes, its symplectic normal form and its parametrised *box relations* — pushing a gate through a normal box yields an updated box and residual *dirty* gates — reappear below, with Theorem 4.4’s reduction to eighteen relations, a step the paper already delegated to Agda. The one major divergence is the step that promotes symplectic to projective completeness (its Proposition 4.9): the paper corrects each rule $C_l = C_r$ to $C_l = C_r ; P$ for the unique Pauli P making it projectively sound and replays the symplectic rewrites, pushing each correction to the end, where it becomes “a potentially different Pauli”. We did not see how to verify it at reasonable cost. We instead work in a syntax where Paulis are genuinely trivial — symplectic circuits have only H , S , CZ and denote $\text{Sp}(2n, \mathbb{Z}_p)$ itself — and reintroduce Paulis and scalars by construction (Figure 2 in the appendix): the Pauli bookkeeping of Proposition 4.9 is done once, by a semidirect-product theorem proved for arbitrary groups before this project began.

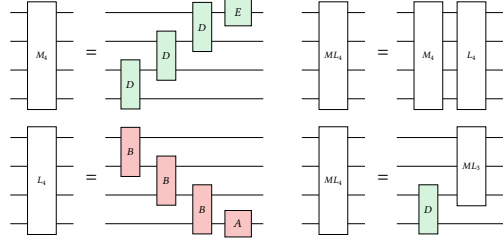
3 The Symplectic Factor

$\text{Sp}(2n, \mathbb{Z}_p)$ is modelled as the group of $\mathbb{Z}_p$ -linear, form-preserving bijections of the phase space Pauli $n = \text{Vec}(\mathbb{Z}_p \times \mathbb{Z}_p)$ n (the exponents of $Z^a X^b$ up to phase, per wire). Against this semantics, completeness rests on the normal form, which we refine into a doubly inductive datatype:

```

ML ML : ℕ → Set
M 0 = ⊤
M (1+ n) = Vec D n × E
L 0 = ⊤
L (1+ n) = Vec B n × A
ML 0 = ⊤
ML 1 = M 1 × L 1
ML m@(2+ n) = M m × L m
              ⊔ D × ML (1+ n)

```

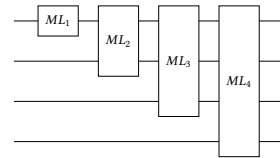


An M -row is a column of D boxes ending in an E box (the paper’s X -normal layer); an L -row is a column of B boxes ending in an A box (its Z -normal layer, with the A box at the bottom). An ML box at width $m \geq 2$ is *either* a full pair of rows *or* a bottom D box followed by an ML box one wire up: a spine of D boxes capped by a pair of rows. This is the first induction, on the width of a single coset representative. The second induction is the staircase:

```

NF : ℕ → Set
NF 0 = ⊤
NF (1+ n) = NF n × ML (1+ n)
[ ] : ∀ {n} → NF n → Word (Gen n)
[ ] {0}    tt    = ε
[ ] {1+ n} (nf , ml) = [ nf ] ↑ • [ ml ]m

```



The table. The coset table is the paper’s “push a gate through a normal box” as a structurally recursive function `ract` (5 600 lines), with its soundness law `ract-sound`:

```

ract      : ML (1+ n) → Gen (1+ n) → Circuit n × ML (1+ n)
ract-sound : ∀ (c : ML (1+ n)) g → let (b' , c') = ract c g in [ c ]c • [ g ]w ≈ b' ↑ • [ c' ]c

```

Each clause of `ract-sound` is derived from the fifteen simplified symplectic relations, drawn in Figure 1; for instance, pushing S through an E box is one S -power step, $[b]^e \bullet S = S^{-b} \bullet S \approx$

$S^{-(b-1)} = [b-1]^e$. A circuit is rewritten to its normal form by repeating this step, gate by gate; every rewrite step is thereby traced to a relation.

Uniqueness. The symplectic group acts on $\text{Vec Pauli}_1 n$, a $2n$ -dimensional symplectic $\mathbb{Z}_p$ -vector space in which Pauli_1 is a two-dimensional symplectic subspace, represented by a wire in a circuit. The elements fixing the bottom wire's subspace pointwise form a copy of $\text{Sp}(2(n-1), \mathbb{Z}_p)$, acting above the bottom wire, and the ML boxes represent its cosets: every $f \in \text{Sp}(2n, \mathbb{Z}_p)$ factors as $f = f' \circ c_n$ with $c_n \in ML_n$ and f' above the bottom wire, and c_n is unique because the bottom-wire output determines the box. Recursing on f' writes f uniquely as $c_1 \circ c_2 \circ \dots \circ c_n$ with $c_k \in ML_k$, which is Theorem 2.2.

Surjectivity. Every symplectic transformation is the action of some circuit, proved constructively by “clear the bottom wire, recurse on the tail” — the paper's Lemma 3.7, its coset box computed rather than asserted:

Theorem-LM : $\forall \{n\} (p q : \text{Pauli } n) \rightarrow \text{sform } p q \equiv \text{true} \rightarrow \exists \lambda (lm : ML \ n) \rightarrow \text{act } [lm]^m \ p \equiv pZ_0 \times \text{act } [lm]^m \ q \equiv pX_0$

Completeness. Circuits with the same denotation rewrite through the table to their normal forms, which denote the same map by soundness and so are equal by Theorem 2.2: the circuits are equivalent. That argument is the framework's by-normalization (Appendix B); with surjectivity, the simplified relations present the symplectic group:

simplified-presentation : $\forall n \rightarrow (n \text{ SRel_} _ == _) \text{ IsPresentationOf } (\text{Sp-group } n)$

4 Composing: Paulis, the Semidirect Product, and Scalars

$\overline{\mathcal{P}}_n \cong (\mathbb{Z}_p \times \mathbb{Z}_p)^n$ is presented over X, Z on each wire by $X^p = Z^p = \varepsilon$ and $ZX = XZ$; its semantic group is the phase space that $\text{Sp}(2n, \mathbb{Z}_p)$ acts on, and its normal form is $X^a Z^b$ per wire. Gluing this to the symplectic presentation of §3 is the semidirect example of Appendix B.5 with $\text{Sp}(2n, \mathbb{Z}_p)$ for S_n and $\overline{\mathcal{P}}_n$ for $(\mathbb{Z}_N)^n$: the presentation has both factors' relations plus the conjugation relations $h n h^{-1} = \text{conj } h n$, the action conj spelling out the paper's Lemmas 2.24–2.25. The theorem's two conditions — the action respects the relations of each factor — are here proved by semantics: the conjugates of congruent words have equal denotations by symplectic soundness, hence are congruent by Pauli *completeness* (respects- Δ ; the paper's “Pauli complete” rules, Definition 4.7). Every premise being a theorem, the semidirect presentation of $\text{Pauli} \times \text{Sp}$ is unconditional.

Plumbing. The semidirect presentation has generators X, Z, H, S, CZ and $3 + 15 + 14$ relations (Pauli, symplectic, conjugation); the paper's rule sets have H, S, CZ alone. Three isomorphisms of presentations close the gap (Figure 2), through *Simplified-V1* (nineteen rules), *Paper-V0* (sixteen) and *Paper-V1* (Theorem 2.1's fifteen): generator reduction (X and Z become words) and relation reduction (the Pauli and conjugation rules become theorems). *Semidirect to Simplified-V1* is an isomorphism on *different* alphabets, f sending S to $R = S \bullet Z^{1/2}$ and h sending S to $Z^{-1/2} \bullet S$; in particular, f induces the embedding of the symplectic group into the projective Clifford group. *Simplified-V1*'s nineteen rules are the fifteen simplified symplectic ones plus $(SH)^3 = \varepsilon$, $H^2 SH^2 S = SH^2 SH^2$ and the two rules pushing X through CZ ; f -well-defined derives from them the images under f of the three Pauli and the fourteen conjugation rules (6 000 lines). *Simplified-V1 to Paper-V0* is the identity on circuits: ten axioms are shared, and *V0*'s Lemmas (4 750 lines) derives c10–c15 from its own axioms. *Paper-V0 to Paper-V1* reaches the paper's own rules. The two rule sets differ in how the multiplier is spelled as a word: *Paper-V0* spells $M_x = R^x H R^{x^{-1}} H R^x H$ over R ; the paper's (T1) spells $M_x = \text{SHS}' x^{-1} x$ over S , with $\text{SHS}' a b = Z^{(b-1)/2} X^{(1-a)/2} S^b H S^a H S^b H$ and semi-MR as $M_g S^{g^2} Z^{(g^2-g)/2} = S M_g$. *Paper-V1* restates the four multiplier rules over SHS' and

drops one: M_1 is now the word $(SH)^3$, so M-power at $k = 0$ already gives $(SH)^3 \approx \varepsilon$, whereas over R it is $(RH)^3$, a different word, and the axiom had to stay. The bridge $R^a HR^b HR^a H \approx SHS'$ a b (Shared.PauliBase) carries the multiplier rules across, giving Theorem 2.1.

Scalars. extension-presentation extends the projective presentation — Paper-V0's, which has order-SH as an axiom — by the scalar (Figure 3): add ω and $\omega^p = \varepsilon$, let ω commute with every gate, and *correct* each rule by its scalar. The correction is ε except on order-SH, the one rule failing on the nose: $(SH)^3 = \omega^{-1/8}$, exponent $(p^2-1)/8 \equiv -1/8$ in $\mathbb{Z}_p$. The multiplier words now denote $|j\rangle \mapsto |xj\rangle$ only up to a scalar. The semantic extension is the Weyl cocycle on the natural semidirect product, $c((P, S), (Q, S')) = \frac{1}{2} \text{sform}(P, S \cdot Q)$, with $S \cdot Q$ the symplectic part acting on the Pauli vector and sform the symplectic form of the phase space, the exponent in $PQ = \omega^{\text{sform}(P,Q)} QP$; its cocycle identity is, by bilinearity, symplectic invariance of sform , and the presented group's cocycle is the pullback of c along Theorem 2.1's isomorphism, so the two agree by construction. Unconditionally:

```
presentation-exact : ∀ n → (n Exact_==_) IsPresentationOf (PE.Presented-group n (exact-generator-data n))
```

The extension is provably non-split.

5 Discussion, Related Work and Conclusion

Everything above is checked by Agda 2.8 with standard library 2.4 under `-safe`: no postulates, no holes. The development is 106 388 lines in 329 files (Table 1): framework 15 000, qubit-specific 60 000 (symplectic 35 000, composition 23 000).

Not formalised are the two well-known *glue facts*: that the projective Clifford group is $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$, and that the Clifford group is a central extension of it by $\langle \omega \rangle$ [Bian et al. 2026a]; no matrix appears in the development.

We have used versions of Claude to improve the language of this paper and to help with the formalisation: composing box relations to prove `ract-sound`, defining the symplectic group, and establishing the equivalence between the code's relations and the qubit paper's Figure 1 modulo scalars; `presentation-exact` was done by Claude.

Related work. The completeness proofs for these gate groups, from Selinger's n -qubit Clifford presentation to the qupit rules, are pen-and-paper (§1); mechanised completeness is rare: Π as a free symmetric rig groupoid [Choudhury et al. 2022] and its square-root extension [Carette et al. 2024] are coherence arguments. Verified compilers and logics [Chareton et al. 2021; Hietala et al. 2021; Lehmann et al. 2025; Li et al. 2022; Paykin et al. 2017; Peduri et al. 2025; Zhou et al. 2023] prove soundness of rewrites, never completeness, and `mathlib` [The `mathlib` Community 2020, 2026], the AFP [Breitner 2010; Kharim et al. 2023] and `MathComp` [Gonthier et al. 2013] have no verified coset enumeration against a given concrete group.

Next: a verified matrix semantics over $U_{p^n}(\mathbb{Z}[1/p, \omega])$, which would close the chain and discharge the glue facts; the qubit case [Selinger 2015], which the odd-prime development does not cover; the qutrit rules of [Li et al. 2025], to be reached from ours at $p = 3$; and submission to the Agda standard library.

References

- Anonymous. 2026a. An Agda library for normal forms and complete relations for circuits. Supplementary material.
- Anonymous. 2026b. Normal Forms and Complete Relations for Circuits in Agda. Under submission; provided as supplementary material.
- Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026a. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions. Manuscript, 2026. Agda code at <https://github.com/onestruggler/acir/releases/tag/v1.0-quantum-submission>.

- Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026b. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions: Derived Relations and Supplemental Proofs. Supplement to [Bian et al. 2026a], 2026; an ancillary file on that paper's arXiv page.
- Joachim Breitner. 2010. Free Groups. Archive of Formal Proofs, <https://www.isa-afp.org/entries/Free-Groups.html>.
- Jacques Carette, Chris Heunen, Robin Kaarsgaard, and Amr Sabry. 2024. With a Few Square Roots, Quantum Computing Is as Easy as Π . *Proceedings of the ACM on Programming Languages* 8, POPL (2024). arXiv:2310.14056.
- Christophe Charetton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. 2021. An Automated Deductive Verification Framework for Circuit-Building Quantum Programs. In *Programming Languages and Systems: 30th European Symposium on Programming (ESOP 2021) (Lecture Notes in Computer Science, Vol. 12648)*. 148–177.
- Vikraman Choudhury, Jacek Karwowski, and Amr Sabry. 2022. Symmetries in Reversible Programming: From Symmetric Rig Groupoids to Reversible Programming Languages. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 6:1–6:32.
- Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyyev, Enrico Tassi, and Laurent Théry. 2013. A Machine-Checked Proof of the Odd Order Theorem. In *Interactive Theorem Proving (ITP 2013) (Lecture Notes in Computer Science, Vol. 7998)*. 163–179.
- Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A Verified Optimizer for Quantum Circuits. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 37:1–37:29.
- Derek F. Holt, Bettina Eick, and Eamonn A. O'Brien. 2005. *Handbook of Computational Group Theory*. Chapman & Hall/CRC.
- Aabid Seeyal Abdul Kharim, Turga Venkata Hanumantha Prathamesh, Shweta Rajiv, and Rishi Vyas. 2023. Formalizing Free Groups in Isabelle/HOL: The Nielsen-Schreier Theorem and the Conjugacy Problem. In *Intelligent Computer Mathematics (CICM 2023) (Lecture Notes in Computer Science, Vol. 14101)*.
- Adrian Lehmann, Ben Caldwell, Bhakti Shah, and Robert Rand. 2025. VyZX: Formal Verification of a Graphical Quantum Language. *ACM Transactions on Programming Languages and Systems* (2025). arXiv:2311.11571.
- Liyi Li, Finn Voichick, Kesha Hietala, Yuxiang Peng, Xiaodi Wu, and Michael Hicks. 2022. Verified Compilation of Quantum Oracles. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 146:1–146:27. doi:10.1145/3563309
- Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A Complete and Natural Rule Set for Multi-Qutrit Clifford Circuits. In *Proceedings of the 22nd International Conference on Quantum Physics and Logic (QPL 2025) (Electronic Proceedings in Theoretical Computer Science, Vol. 426)*. 23–78. arXiv:2508.14670.
- Ulf Norell. 2007. *Towards a Practical Programming Language Based on Dependent Type Theory*. Ph. D. Dissertation. Chalmers University of Technology.
- Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017)*. 846–858.
- Anurudh Peduri, Ina Schaefer, and Michael Walter. 2025. QbC: Quantum Correctness by Construction. *Proceedings of the ACM on Programming Languages* 9, OOPSLA1 (2025). doi:10.1145/3720433
- Peter Selinger. 2015. Generators and relations for n -qubit Clifford operators. *Logical Methods in Computer Science* 11, 2:10 (2015). arXiv:1310.6813.
- Charles C. Sims. 1994. *Computation with Finitely Presented Groups*. Encyclopedia of Mathematics and its Applications, Vol. 48. Cambridge University Press.
- The Agda Community. 2026. The Agda standard library, version 2.3. <https://github.com/agda/agda-stdlib>.
- The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. 367–381.
- The mathlib Community. 2026. Mathlib documentation: PresentedGroup, CoxeterGroup, and the Nielsen-Schreier theorem. https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/FreeGroup/NielsenSchreier.html.
- Li Zhou, Gilles Barthe, Pierre-Yves Strub, Junyi Liu, and Mingsheng Ying. 2023. CoqQ: Foundational Verification of Quantum Programs. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 29:1–29:33.

A The Simplified and Exact Rules and the Size of the Development

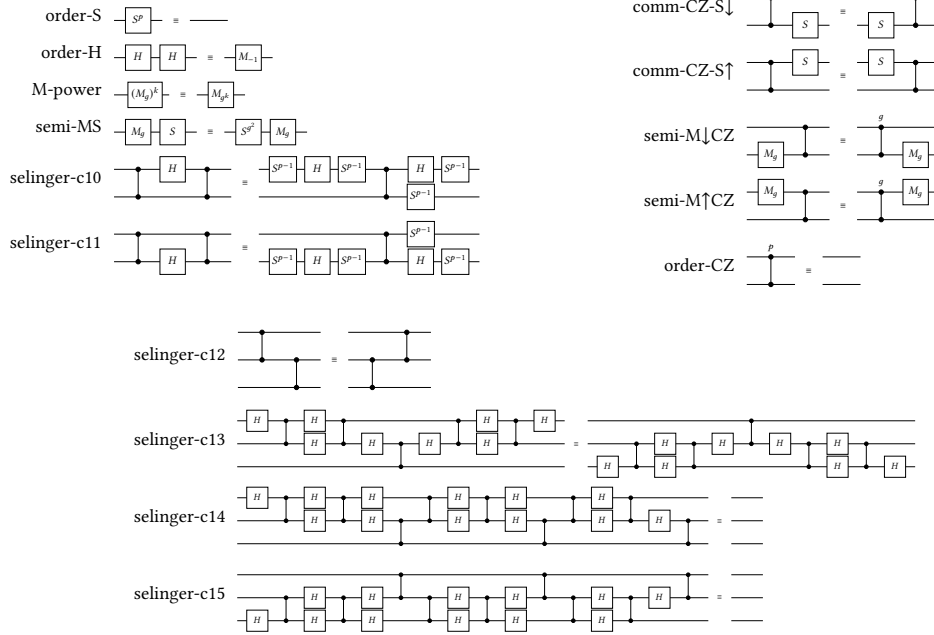


Fig. 1. The fifteen simplified symplectic rules (§3) as circuit equations, drawn from the Agda terms; M_g is the multiplier of a primitive root g , and selinger-c10–c15 are Selinger’s two- and three-wire rules.

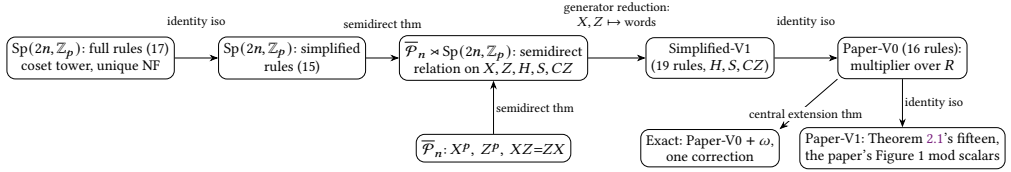
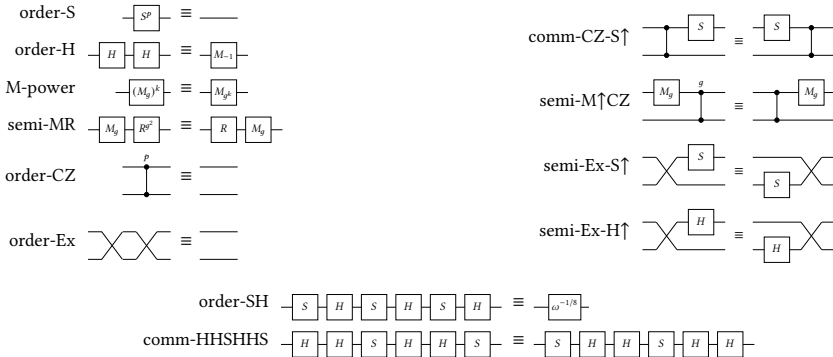


Fig. 2. The chain of presentations: every arrow is a verified presentation theorem or isomorphism of presentations; the semantic group is $\text{Sp}(2n, \mathbb{Z}_p)$ on the left, $\overline{\mathcal{P}}_n \rtimes \text{Sp}(2n, \mathbb{Z}_p)$ from the semidirect level on, and a central extension of it by $\langle \omega \rangle$ at “Exact”.



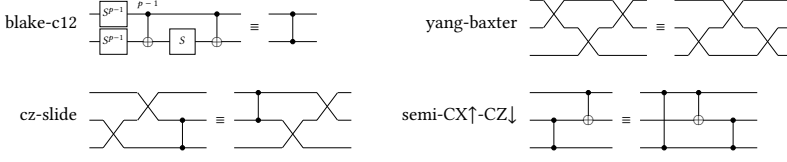


Fig. 3. The gate rules of the exact rule set `_Exact,===_` (§4) as circuit equations, drawn from the Agda terms: Paper-V0’s sixteen with the one correction, order-SH becoming $(SH)^3 = \omega^{(p^2-1)/8}$. Not drawn are the scalar’s own rules, $\omega^p = \varepsilon$ and ω commuting with every gate. Diagrams read in matrix-multiplication order (§2).

Table 1. Size of the development (lines of Agda incl. comments, wc -l, September 2026).

Component	Files	Lines
Framework (Word, Presentation, Circuit, Normalization, ForStdlib)	54	14 848
Symplectic group (normalisation 8 352; box relations 18 161; soundness etc. 8 886)	118	35 399
Projective Pauli group	3	1 220
Projective Clifford: semidirect, plumbing, shared calculus	26	17 996
Clifford with scalars	11	5 029
Other examples in the tree (symmetric, cyclic, semidirect, Clifford+T, qubit Clifford)	107	27 552
Not on the chain (Simplified-V2), index, notations	10	4 344
Total	329	106 388

B An Introduction to the Framework

The formalisation is built on a library for presentations, normal forms and completeness of circuit relation sets [Anonymous 2026a,b]. This appendix introduces it on its smallest complete case study: circuits built from a single two-wire gate, the swap σ , which present the symmetric groups. The walkthrough exhibits the pipeline — gates, relations, cosets, tower, semantics, uniqueness, completeness, presentation — that §3 instantiates again for the symplectic factor, concretely enough that the latter can be read as the same pipeline with richer gates and a richer action; §4 glues such presentations together by the library’s product constructions.

B.1 Gates, generators, and circuits

A gate set is a family $\text{Gate} : \mathbb{N} \rightarrow \text{Set}$ giving, for each arity, the gates on that many adjacent wires; for permutations there is one gate, on two wires. The generic circuit layer of the library turns any such family into wire-indexed *generators*: a gate placed at the bottom of the wires, or a generator shifted up by one wire.

```

data Gate : ℕ → Set where
  σ-gate : Gate 2
data Gen : ℕ → Set where
  gate₁ : Gate 1 → Gen (1 + n) — a 1-ary gate on the bottom wire
  gate₂ : Gate 2 → Gen (2 + n) — a 2-ary gate on the bottom two wires
  ↑ : Gen n → Gen (1 + n) — the same generator, one wire higher

```

Here $1 + n$ and $2 + n$ are the library’s patterns for $1 + n$ and $2 + n$, and n is an implicit argument, so gate_2 σ -gate is a generator at any width from two upward. An n -wire *circuit* is a word of generators, $\text{Circuit } n = \text{Word } (\text{Gen } n)$: a single generator $[g]^w$, the empty circuit ε — n bare wires, the identity circuit at that width — or a sequential composition $c \bullet c'$. We write σ for the one-letter circuit $[\text{gate}_2 \text{ } \sigma\text{-gate}]^w$ and $c \uparrow$ for c shifted up one wire ($\uparrow = \text{wmap } \uparrow$). Read diagrammatically, with

wires drawn bottom-up and a word drawn left to right as written, σ is a crossing of the bottom two wires, $\sigma \uparrow$ the same crossing one wire higher, and $\sigma \bullet \sigma \uparrow \bullet \sigma$ the three-wire braid on the left of the yang-baxter equation drawn beside its axiom below.

B.2 Relations: two axioms, structure for free

The group-specific relations are an inductive family indexed by the wire count. Each axiom is stated once, with its gates at the bottom of the wires and the width left open (the implicit n), so that one constructor covers every width at which the axiom fits; the structural rule $\text{cong}\uparrow$ below moves it to any height. (Agda names may be mixfix: each underscore in $_S\text{Rel},_===_$ marks an argument position, so the family is applied as $n \text{ SRel}, w === v$, read “ $w = v$ is an axiom on n wires”.)

```
data _SRel, _===_ : (n : ℕ) → WRel (Gen n) where
```

```
  order : (2+ n) SRel,  $\sigma \bullet \sigma === \varepsilon$ 
```

```
  yang-baxter : (3+ n) SRel,  $\sigma \bullet \sigma \uparrow \bullet \sigma === \sigma \uparrow \bullet \sigma \bullet \sigma \uparrow$ 
```

These two axioms are two thirds of the Coxeter presentation of S_n : $\sigma^2 = \varepsilon$ and the braid (Yang–Baxter) relation. The third Coxeter relation, that swaps on disjoint wires commute, is not specific to permutations: it is one of the relations every circuit theory needs, whatever its gates. By a *circuit theory* we mean a gate set with gate-specific relations, presenting for each width n a monoid of n -wire circuits under sequential composition. Such circuits are the fixed-width slices of a symmetric monoidal category (a PROP), and the monoidal structure leaves exactly two traces at fixed width: the shift $c \mapsto c \uparrow$ is “tensoring with an identity wire”, so equations must be preserved by it, and the interchange law says that gates on disjoint wires commute. (The library also admits 0-ary gates — global scalars, available at every width — with one more structural rule, that a scalar is identified with its own shift, $\omega \uparrow = \omega$; that it then commutes with every generator is a theorem, once scalars are assumed to commute with one another, which is vacuous for a single scalar.) These are the structural rules every circuit theory shares — what a pen-and-paper presentation leaves implicit in “a congruence compatible with composition and tensor product” — and the circuit layer adds them generically: Lift-Relation extends any gate-specific family to the full relation set $_V\text{Rel},_===_$:

```
data _VRel, _===_ : (n : ℕ) → CRel n where
```

```
  srel : n SRel, w === v → n VRel, w === v
```

```
  cong↑ : n VRel, w === v → (1+ n) VRel, w ↑ === v ↑
```

```
  comm1 : (h : Gate 1) (g : Gen n) → (1+ n) VRel, [g↑]w • [gate1 h]w === [gate1 h]w • [g↑]w
```

```
  comm2 : (h : Gate 2) (g : Gen n) → (2+ n) VRel, [g↑↑]w • [gate2 h]w === [gate2 h]w • [g↑↑]w
```

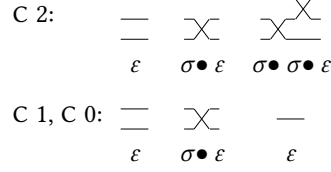
From here on, $\approx$ denotes the monoid congruence generated by $n \text{ VRel},_===_$: the least relation containing the axioms and closed under reflexivity, symmetry, transitivity, congruence of $\bullet$, associativity, and the unit laws; deciding it is the word problem. Because σ is its own inverse and shifts of invertible generators are invertible, the presented monoid is a group; we witness this by a Grouplike proof (every generator has a left inverse), by induction on generators.

B.3 Cosets and the staircase normal form

The engine behind the normal form is coset enumeration along the chain $S_1 < S_2 < \dots < S_n < S_{n+1}$, where S_n sits inside S_{n+1} as the permutations fixing the bottom wire — syntactically, the circuits of the form $c \uparrow$. Read as permutations, two circuits lie in the same right coset of S_n exactly when they bring the same wire to the bottom position; there are $n+1$ choices, so S_n has index $n+1$, and a natural representative of each coset is the *descending staircase* of swaps that carries that wire down. In the library a coset is not a set of circuits but a label naming one, an element of the finite type $C \ n$; here the label is the length of the staircase, as a unary numeral bounded by the width, and the

section $[_]^c$ maps it to the staircase circuit. The representatives on three, two, and one wires are drawn on the right:

data $C : \mathbb{N} \rightarrow \text{Set}$ where
 $\varepsilon : C \ 0$
 $\sigma \bullet _ : C \ n \rightarrow C \ (1_+ \ n)$
 $[_]^c : C \ n \rightarrow \text{Circuit} \ (1_+ \ n)$
 $[\varepsilon]^c = \varepsilon$
 $[\sigma \bullet c]^c = \sigma \bullet ([c]^c \uparrow)$



So $C \ n$ has exactly the $n+1$ elements drawn above for $n = 2, 1, 0$, matching the index of S_n in S_{n+1} . Why this yields a normal form is the classical argument of computational group theory [Holt et al. 2005; Sims 1994]: every $f \in S_{n+1}$ lies in one coset, say with representative c_n , so $f = f' \cdot c_n$ with $f' \in S_n$, and recursing on f' writes f uniquely as $c_1 c_2 \cdots c_n$ with $c_k \in C \ k$ – a numeral in the factorial number system, one digit per level, since $|S_{n+1}| = (n+1) \cdot n \cdots 2$.

Computing this factorisation is the job of the coset table. Given a coset, represented by its staircase, and one generator, the table says where their product lands – the updated coset – and what is left over on the wires above – the residual circuit – so that staircase followed by generator equals residual (shifted up) followed by updated staircase. Pushing σ into a staircase can lengthen it, cancel with it ($\sigma^2 = \varepsilon$), pass it through and shift it up (Yang–Baxter), or commute past; for a generator that does not touch the bottom wire we can recurse. The residual is empty or a single swap here, but in richer gate sets it can be a longer circuit, which is why the table returns a circuit on one wire fewer rather than a generator (Figure 4 draws the four cases):

$\text{ract} : C \ (1_+ \ n) \rightarrow \text{Gen} \ (2_+ \ n) \rightarrow \text{Circuit} \ (1_+ \ n) \times C \ (1_+ \ n)$
 $\text{ract } \varepsilon \ \sigma\text{-gen} = \varepsilon, \sigma \bullet \varepsilon$ – grows
 $\text{ract } (\sigma \bullet \varepsilon) \ \sigma\text{-gen} = \varepsilon, \varepsilon$ – cancels ($\sigma^2 = \varepsilon$)
 $\text{ract } (\sigma \bullet \sigma \bullet c) \ \sigma\text{-gen} = \sigma, \sigma \bullet \sigma \bullet c$ – passes (Yang–Baxter)
 $\text{ract } \varepsilon \ (g \uparrow) = [g]^w, \varepsilon$ – commutes past
 $\text{ract } (\sigma \bullet c) \ (g \uparrow) = \text{proj}_1(\text{ract } c \ g) \uparrow, \sigma \bullet (\text{proj}_2(\text{ract } c \ g))$ – recurses

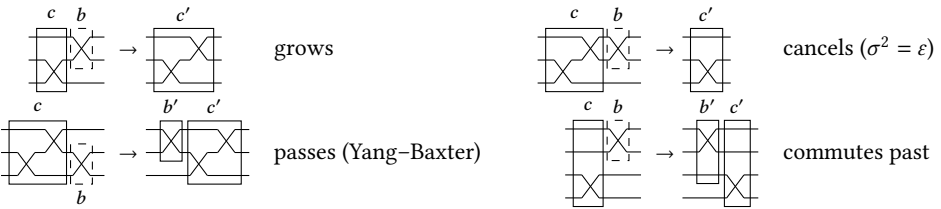
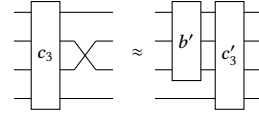


Fig. 4. Pushing a swap through a staircase: the four cases of the coset table ract as circuit rewrites.

σ -gen abbreviates gate₂ σ -gate. The last clause is the recursion: a shifted generator meets a staircase by meeting its tail one wire down. Each clause is certified by the *soundness law*, pictured on the right and proved once per table by a short equational derivation from the axioms:

$\text{ract-sound} : \forall \ c \ b \rightarrow \text{let } (b', c') = \text{ract } c \ b \text{ in } [c]^c \bullet [b]^w \approx b' \uparrow \bullet [c']^c$

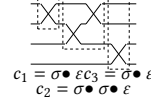
The stateful word traversal $_{}^t$ extends ract to whole circuits, threading the coset from left to right and concatenating the residuals; run from the identity coset, it maps an $(n+2)$ -wire circuit to an $(n+1)$ -wire circuit and a coset – one level of normalisation.

Iterating the level is the coset tower, whose carrier grows by one digit per level:

```

491  NF : ℕ → Set
492  NF 0 = T
493  NF (suc k) = NF k × C k - T × C 0 × C 1 × ... × C k

```



so $\text{NF } n$ has exactly $n!$ elements. The normal-form function $\text{nf} : \text{Circuit } n \rightarrow \text{NF } n$ runs the tables; its *section* $\text{inv-nf} : \text{NF } n \rightarrow \text{Circuit } n$ rebuilds a circuit from its digits by concatenating the staircases, suitably shifted, as drawn on the right for $(\text{tt}, c_0, c_1, c_2, c_3) \in \text{NF } 4$, the digit $c_0 \in \text{C } 0$ being the empty staircase — one staircase per level, each shifted up to the wires of its level: a “staircase of staircases”.

B.4 Semantics, uniqueness, and completeness

Everything above is syntax. The development interprets circuits in the group $\text{Permutation}'\text{-group } n$ of permutations of $\text{Fin } n$, with σ swapping the bottom two elements. *Soundness* — $w \approx v$ implies $[[w]] \doteq [[v]]$ — is a small case analysis over the axioms. The heart of the matter is the *unique normal form* property, that normal forms with equal denotations are equal: $[[\text{inv-nf } u]] \doteq [[\text{inv-nf } v]]$ implies $u \equiv v$. It is proved by induction on the tower: the lower digits fix the last point, so the top digit alone determines where the permutation sends it; equal denotations therefore force equal top digits, and cancelling them reduces the claim to the level below. Soundness and uniqueness are exactly the premises of the generic lemma by-normalization, which concludes *completeness*: $[[w]] \doteq [[v]]$ implies $w \approx v$. Together with surjectivity (every permutation is a product of adjacent transpositions), these assemble into the library’s *presentation record*: a Grouplike witness, which makes the presented monoid a group, and a proof that $[[_]]$ is a group isomorphism from that group onto the target ($\text{IsGroupIsomorphism}$ is the standard library’s, between the group the witness induces and G ; the record’s module plumbing that says so is elided):

```

514  record _IsPresentationOf_ (___ : WRel X) (G : Group a ℓ) : Set (a ⊔ ℓ) where
515    field
516      gl : Grouplike ___
517      [[_]] : Word X → Group.Carrier G
518      iso : IsGroupIsomorphism [[_]]

```

Every presentation theorem in the paper has the shape of this one:

```

520  symmetric-presentation : ∀ n → (n VRel, ___) IsPresentationOf (Permutation'-group n)

```

read: the monoid of swap circuits modulo $\{\text{order, yang-baxter}\}$ and the structural rules *is* the group of permutations of $\text{Fin } n$. The symplectic factor of §3 is this pipeline again, with richer gates and a richer action, and §4 composes such presentations.

B.5 The semidirect product $(\mathbb{Z}_N)^n \rtimes S_n$

Fix a base order $N = m+1$. The cyclic presentation $\langle T \mid T^N = \epsilon \rangle$, iterated by the n -fold direct-product construction, presents $(\mathbb{Z}_N)^n$ over n copies of T , one per coordinate; the other factor is already in hand: the swap circuits present S_n (§B.4’s symmetric-presentation). The semidirect product $(\mathbb{Z}_N)^n \rtimes S_n$ is S_n acting on the coordinates, and the action is specified *syntactically*, generator by generator: the bottom swap exchanges the bottom two coordinates, and a shifted swap acts on the shifted coordinates and fixes the bottom one.

```

533  act : ∀ {n} → Gen n → T ⊔^ n → T ⊔^ n
534  act (gate2 σ-gate) x = sw2 x
535  act {2, n} (g ↑) (inj1 tt) = inj1 tt
536  act {2, n} (g ↑) (inj2 y) = inj2 (act g y)

```

The construction takes the action *word-valued* — in richer examples a conjugated generator becomes a whole word — so it is fed act read as a one-letter word, $\text{conj } g \ x = [\text{act } g \ x]^w$. The semidirect

theorem asks for two compatibility lemmas — that the action respects the relations of each factor. With the two factor presentations and these lemmas in hand, the theorem gives

`sd-pres : ∀ n m → (((suc m Cn, _ == _) ⊕ ^ n) ⋄ (n VRel, _ == _) ⋄ ConjRelw conj) IsPresentationOf (semidirect-group n m)`

read: the relations fall into three groups — the cyclic relations, one copy per coordinate plus the commutations between distinct coordinates; the swap-circuit relations of S_n ; and the conjugation relations ConjRel^w , one per swap and coordinate generator, spelling out the action — and, joined by $\diamond$, they are together a presentation of the semidirect product $(\mathbb{Z}_N)^n \rtimes S_n$.